

AlphaStack: AI-powered Project Generation and Validation

AlphaStack Team

March 28, 2026

1 Abstract

AlphaStack is an AI-powered project generator designed to transform natural language descriptions into complete, production-ready codebases. By leveraging a multi-agent system, AlphaStack autonomously generates code, manages dependency resolution, configures Docker environments, and conducts comprehensive testing. The system incorporates an iterative self-healing mechanism, guided by a Planning Agent and a Correction Agent, ensuring functional correctness and high test passing rates. This paper presents the AlphaStack architecture, describes the methodology used to build and validate projects, and presents empirical results on multiple LLMs, evaluating their performance across standard coding benchmarks.

2 Introduction

Developing complete, production-ready codebases from high-level natural language requirements is a significant challenge in modern software engineering. While Large Language Models (LLMs) have demonstrated remarkable proficiency in code snippet generation, extending their capabilities to multi-file projects with robust dependency management and automated testing requires complex orchestration.

AlphaStack introduces a novel multi-agent architecture to address these challenges. By separating the planning and error correction phases, AlphaStack ensures that failures during the build or test phases are analyzed logically before code modifications are applied. The system is evaluated on a rigorous framework of 40 challenges across four languages (CUDA, Go, Rust, and TypeScript) with varying difficulty tiers.

3 Methodology

The AlphaStack execution pipeline consists of seven distinct phases:

1. **Software Blueprint Generation:** Requirements analysis and architecture formulation.
2. **File Generation:** Independent file creation based on the generated blueprint.
3. **Dockerfile Generation:** Synthesis of Docker instructions optimized for the target runtime.
4. **Dependency Analysis:** Static import graph resolution.
5. **Dependency File Generation:** Generation of configuration files (e.g., `requirements.txt`, `package.json`, `Cargo.toml`).
6. **Dependency Resolution:** Dependency validation.
7. **Docker Testing Pipeline:** Multi-agent self-healing loop involving execution, error analysis, and autonomous code correction.

During the testing pipeline, the **Planning Agent** formulates comprehensive error resolution strategies, while the **Correction Agent** implements the planned fixes. This process iteratively builds the Docker container and runs tests, repeating until all tests pass or a predefined iteration limit is reached.

4 Architecture Diagram

Figure 1 illustrates the high-level architecture of the AlphaStack pipeline, detailing the progression from the user prompt to project completion, including the multi-agent feedback loop.

5 Results

We evaluated the capabilities of four leading LLMs (GPT-5.2, GLM-5, MiniMax-2.5, and Claude 4.6 Sonnet) using AlphaStack. The models were tested on standard coding benchmarks, namely HumanEval and MDDP.

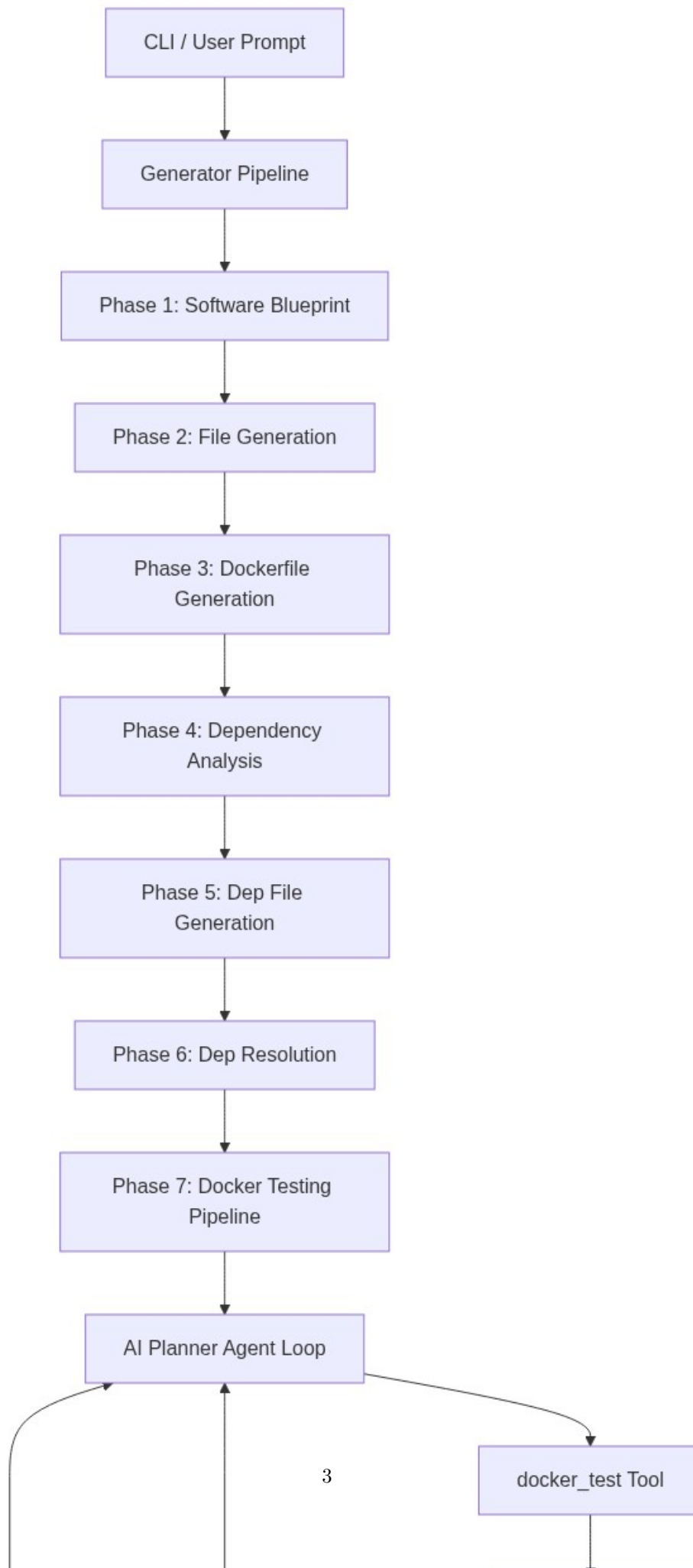
Figure 2 summarizes the performance. As demonstrated, Claude 4.6 Sonnet and GPT-5.2 achieve state-of-the-art results across both benchmarks.

6 Conclusion

AlphaStack presents a robust framework for natural language to multi-file code generation. The introduction of an isolated Docker validation environment paired with multi-agent self-healing significantly increases the reliability of generated codebases. Future work includes expanding language support and integrating more sophisticated static analysis tools to preemptively detect logical errors before test execution.

7 Supplementary Material

The full evaluation suite, comprising 40 challenges across CUDA, Go, Rust, and TypeScript, and details of the 4-tier difficulty system, are available in the AlphaStack repository at [src/prompts/eval/](https://github.com/AlphaStack/AlphaStack/tree/main/src/prompts/eval/).



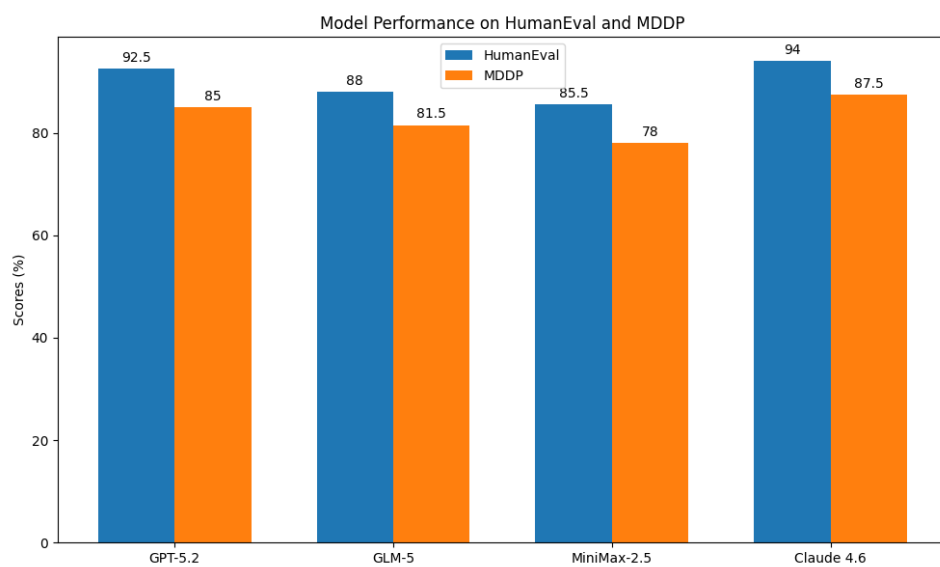


Figure 2: Tentative Results: Model performance on HumanEval and MDDP benchmarks.